

```
import notebooks_pyfile as oop
```

```
help(oop)
```

```

| get_info(self)
| get_vehType(self)
| get_wheels(self)
| set_vehType(self, vehType)
|
| -----
| Data descriptors inherited from Vehicle:
|
| __dict__
|     dictionary for instance variables
|
| __weakref__
|     list of weak references to the object
|
class Vehicle(abc.ABC)
| Vehicle(wheels, make, model, color)
|
| Method resolution order:
|     Vehicle
|     abc.ABC
|     builtins.object
|
| Methods defined here:
|
| __init__(self, wheels, make, model, color)
|     Initialize self. See help(type(self)) for accurate signature.
|
| get_info(self)
|
| get_vehType(self)
|
| get_wheels(self)
|
| move(self, txt='The')
|
| set_vehType(self, vehType)
|
| -----
| Data descriptors defined here:
|
| __dict__
|     dictionary for instance variables
|
| __weakref__
|     list of weak references to the object
|
| -----
| Data and other attributes defined here:
|
| __abstractmethods__ = frozenset({'move'})
|
FILE
/content/notebooks_pyfile.py
```

```
mycar = oop.Car(4, "Toyota", "Corolla", "Red")
mybike = oop.Motorcycle(2, "Honda", "CBR", "Blue")
```

```
info = mycar.get_info()
print(info)
```

```
Wheels : 4, Make : Toyota, Model : Corolla, Color : Red
```

```
mycar.move("My" + info)
```

```
MyWheels : 4, Make : Toyota, Model : Corolla, Color : Red Car is being driven.
```

Start coding or [generate](#) with AI.

▼ Destructor

When we have huge data and we do not want unnecessary memory then we use this

```
class Counter:
    _instances = 0

    def __init__(self, name):
        Counter._instances += 1
        self.name = name
        print(f"Created {self.name}, total instances: {Counter._instances}")

    def __del__(self):
        Counter._instances -= 1
        print(f"Destroyed {self.name}, remaining instances: {Counter._instances}")
```

```
def create_counter():
    c = Counter("Local")

def reassign_counter():
    c1 = Counter("First")
    c2 = Counter("Second")
```

```
print("Testing scenerio 1:")
create_counter()
print("\nTesting scenerio 2:")
reassign_counter()
```

```
Testing scenerio 1:
Created Local, total instances: 13
```

```
Testing scenerio 2:
Created First, total instances: 14
Created Second, total instances: 15
```

▼ Empty Class

```
class MyEmptyClass:
    pass

obj = MyEmptyClass()
obj.name = "Python"
```

▼ DataClass

```
from dataclasses import dataclass

@dataclass
class User:
    id : int
    username : str
    email : str = "default@gmail.com"

user1 = User(1, "Aliyan")
print(user1)

User(id=1, username='Aliyan', email='default@gmail.com')
```

Start coding or [generate](#) with AI.

